name: eshop-automation description: Build a data-driven, multi-browser Playwright suite for one feature of the EShop SUT (eshop-sut), then run it on Chromium/Firefox/WebKit and produce one HTML report per engine carrying "Run by: {StudentID}". Use when asked to automate an FR-xx feature, add browser coverage, or turn manual test cases into Playwright specs for this project.

EShop feature automation

Turns one FR-xx feature of the EShop SUT into a reviewed, data-driven Playwright suite with per-engine HTML reports and a defect list.

This skill encodes what eight debugging cycles on FR-01, FR-09 and FR-14 cost to learn. Follow the order — the expensive mistakes all came from skipping step 2.

Workflow

1. Read the specification BEFORE the code

Open `eshop-sut/api_specification.md` and the feature's UI text. **These, not the implementation, are the source of truth for what a test should expect.**

An AI given the source code will write tests that match current behaviour — it will propose "password `Password123!` → expect rejected" because that is what the regex does. Such a test is always green and can never find a bug: it promotes the defect to a requirement. Expect red tests; each one should map to a defect.

2. Probe the real DOM and the real behaviour — never assume

Write a throwaway script that opens the page and prints what is actually there, then delete it. Check at minimum:

- Does `getByLabel()` resolve? **In this SUT it never does** — labels carry no `htmlFor` and inputs carry no `id` / `name`. Anchor on the wrapper element that holds the label text instead.

- What do the inputs actually look like? The admin login e-mail field has **no `type` attribute at all**, so `input[type="email"]` matches nothing. Its submit button reads `Login`, not `Đăng nhập`.
- What does the feature really do for the interesting inputs? Record the answers; they become both the expected values and the first draft of the bug list.

3. Put every input value in `tests/data/`

The assignment rejects inline literals. Use a CSV for anything matrix-shaped (a rule table, a calculation grid) and a JSON file for grouped cases. Read them with `readCsv()` / `readJson()` from `tests/utils/csv.js` and generate tests in a loop.

4. Write the spec with race-free assertions

Use at least three distinct assertion patterns; six are already established — navigation, web-first text, DOM property, back-end API, element count/state, and invariants. Tag each call site `[P1]`..`[P6]`.

The single most important rule: never prove absence by asserting that the URL did not change. Right after a click the SPA has not resolved its POST, so the URL is unchanged no matter what the server decided, and the assertion goes green instantly. Four tests once passed against a SUT that accepted garbage e-mail addresses. Ask the back end what it stored instead.

Related traps:

- Assert HTML5 `validity.valueMissing`, not the browser's validation message — that string differs per engine and per OS language.
- Before asserting via the API after a UI action, wait for the SPA to settle (`await expect(page).toHaveURL(...)` or `waitForResponse`). A missing wait produced a flake that appeared **only on WebKit**.
- Money is rendered with the browser's locale, which Playwright leaves at en-US: the page shows `50,000`, not `50.000`. Accept both separators when parsing.

5. Make tests independent and self-cleaning

Give every created record a run-unique suffix. **Clean up on both branches** — a negative test leaves a row behind exactly when it catches a bug, and the debris breaks a later test for reasons that have nothing to do with what it checks.

Never assert on seeded data another test may legitimately delete; plant your own row and assert on that.

6. Run per engine, one report each

```
node scripts/run-multibrowser.mjs tests/frXX-<feature>.spec.js
```

Produces `playwright-report/frXX-<feature>-{chromium,firefox,webkit}/`, each with `Run by: {StudentID}` and an ISO timestamp in the header.

7. Verify — counts are not evidence

```
node scripts/verify-report-banner.mjs
```

Then scan every `results.json` and confirm **each failing test is tagged `@bug`**. Matching pass/fail totals prove nothing on their own: a broken helper once made five tests red with `admin login must succeed`, a message unrelated to the defects they targeted, while the totals still looked exactly as expected. Read the failure *reason*, not the count.

Also re-run the whole matrix after any fix. One patch to a money parser fixed a sign error and simultaneously broke six passing tests on all three engines.

8. Record defects

For each red test write a bug entry (steps, expected, actual, evidence) in `23127195/bug-report/BUG_REPORT.md`, capture a screenshot under `23127195/evidence/bugs/`, and open a GitHub issue that embeds the screenshot by its `raw.githubusercontent.com` URL — push first, or the image will not render.

SUT facts worth not rediscovering

Fact	Value
Backend / web / admin	`:3000` / `:5173` / `:5174`
Admin credentials	`admin@eshop.com` / **`Admin123!`** — `setup_guide.md` says `admin123` and is **wrong**
Account lockout	2 wrong passwords lock an account for 180 s, and while locked even the correct password is refused. Cache the admin token; log in once.
Reseed	`npm run sut:seed` (the backend also reseeds on every start)
Vite binding	IPv6 `::1` only — use `localhost`, never `127.0.0.1`
Cart state	Plain React state, no persistence: `page.goto()` empties it. Navigate by clicking for end-to-end flows.
`frontend-admin`	Not covered by `webServer` in the config; start it manually before FR-14-style suites

Deliverables checklist

- ☐ ≥ 12 test cases for the feature
- ☐ All data in `tests/data/*.csv` and `*.json`, nothing inline
- ☐ ≥ 3 assertion patterns, tagged at each call site
- ☐ 3 engines, 3 standalone HTML reports, each showing `Run by: {StudentID}` + ISO timestamp
- ☐ Every failing test tagged `@bug` and mapped 1-to-1 to a bug entry
- ☐ Bug report updated, screenshots captured, GitHub issues opened
- ☐ Review findings recorded: what the AI got wrong and **why**